

Azure Databricks Medallion Lakehouse with Incremental CDC

Beginner Implementation and Troubleshooting Reference

Contents

Section	What you will learn
Part 1	Goal, architecture, services, naming, and prerequisites
Part 2	Manual Azure Portal setup and Unity Catalog configuration
Part 3	Notebook import, Phase 1, Phase 2, validation, and CDF
Part 4	Terraform creation, repeatable execution, and destruction
Part 5	Errors encountered, working fixes, cost controls, and interview review

How to follow the guide

Choose either manual infrastructure setup or Terraform infrastructure setup. After the resources exist, both paths use the same notebooks, sample files, Databricks jobs, validation SQL, and cleanup checks.

POC objective

Build a Bronze, Silver, and Gold Delta Lake pipeline on Azure Databricks and demonstrate incremental processing from new ADLS files.

- Incremental ingestion: Auto Loader processes only files not already recorded in its checkpoint.
- Data quality: invalid orders and customers move to quarantine with an error reason.
- Current and historical dimensions: SCD Type 1 and SCD Type 2 are implemented in Gold.
- Change processing: Delta MERGE applies inserts and updates; Change Data Feed exposes changed rows.
- Governance: Unity Catalog organizes and secures catalog.schema.table objects.
- Repeatability: Terraform creates the disposable lab and destroys it after verification.

Architecture and data flow

Layer	Responsibility
1. ADLS landing	Phase 1 and Phase 2 CSV files arrive under raw/orders and raw/customers.
2. Bronze Delta	Source values are retained with <code>_ingest_ts</code> , <code>_source_file</code> , and <code>_batch_id</code> .
3. Silver Delta	Types, quality rules, standardization, quarantine, and latest-record deduplication are applied.
4. Gold Delta	<code>fact_orders</code> , <code>dim_product</code> , and historical <code>dim_customer</code> are maintained with MERGE.
5. CDF consumer	Only fact-table changes are written to <code>fact_orders_changes_audit</code> .

Key idea

A checkpoint is processing state, not business data. Deleting it can cause files to be discovered again, so checkpoint paths must be protected and unique per stream.

Services used - and services not required

Scope	Details
Used	ADLS Gen2, Azure Databricks, PySpark, Spark SQL, Delta Lake, Auto Loader, Unity Catalog, Databricks Jobs, Terraform
Not used in POC-02	Azure SQL Server, Azure SQL Database, SQL firewall rules, Azure Data Factory, and ADF pipelines
Why	This POC is a lakehouse and CDC exercise. Creating SQL or ADF resources would add cost without supporting the defined architecture.

Avoid confusion

SQL Server and Azure Data Factory belong to the separate secure batch ADF-ADLS-SQL POC. Databricks Jobs provide orchestration for this POC.

Resource naming and storage layout

Resource	Example
Resource group	rg-azde-poc02
Storage account	stazdepoc02<unique>
Container	poc02
Access Connector	ac-azde-poc02
Databricks workspace	dbw-azde-poc02
Catalog and schemas	azde_poc.bronze, silver, gold, quarantine

poc02/
raw/orders/
raw/customers/
checkpoints/
schema/
quarantine/

Prerequisites checklist

- Azure account: permission to create resources and role assignments. Owner is simplest for a personal lab.
- Azure CLI: the az executable must be installed separately from Python Azure SDK packages.
- Terraform: version 1.x available on Windows PATH.
- Project files: Terraform folder, six notebooks, two sample-data phases, SQL validation file, and numbered CMD scripts.
- Regional capacity: enough Databricks VM quota, or a single-node cluster that fits the current limit.
- Safety: do not commit tokens, secrets, terraform.tfstate, private tfvars, or Azure CLI profiles.

Cost guardrail

Use serverless or a small single-node development cluster and set 15-minute auto-termination.

Local authentication and verification

Run these commands in a new Windows CMD session after installing Azure CLI and Terraform.

```
az --version
az login
az account list --output table
az account set --subscription "<subscription-id-or-name>"
az account show --output table
terraform -version
```

Expected result

The selected subscription and tenant are correct, and both az and terraform return versions without command-not-found errors.

Error fix - az is not recognized

Symptom: CMD prints that az is not recognized as an internal or external command.

Root cause: installing azure-identity in Poetry installs a Python authentication SDK. It does not install the Azure CLI executable.

- 1 Install Azure CLI for Windows.
- 2 Close every existing CMD and PowerShell window.
- 3 Open a new CMD and run `where az`.
- 4 Run `az --version` and then `az login`.
- 5 If PATH is still stale, restart Windows or add the Azure CLI installation directory to PATH.

Manual setup - create the resource group

- 1 Open Azure Portal and search for Resource groups.
- 2 Select Create, choose the intended subscription, and enter rg-azde-poc02.
- 3 Choose one region and reuse it for storage, Access Connector, and Databricks.
- 4 Select Review + create, then Create.
- 5 Open the resource group and confirm it is ready before creating child resources.

Region choice

The screenshots show Central India for the completed run. A different region is acceptable when it has enough Databricks compute quota.

Screenshot - resource groups

Home > Resource Manager

Resource Manager | Resource groups

Default Directory

Search

Resource Manager

All resources

Favorite resources

Recent resources

Resource groups

Tags

Organization

Tools

Deployments

Help

Create Manage view Refresh Export to CSV Open query Assign tags Add to service group

Filter for any field... Subscription equals all Location equals all Add filter

☐	Name ↑		Subscription	Location
☐	 NetworkWatcherRG	...	Azure subscription 1	Central India
☐	 rg-azde-poc02	...	Azure subscription 1	Central India
☐	 rg-azde-poc02-databricks-managed	...	Azure subscription 1	Central India

Showing 1 - 3 of 3. Display count: auto

[Give feedback](#)

Add or remove favorites by pressing Ctrl+Shift+F

The POC resource group and the Databricks-managed resource group are visible in Azure Portal. Beginner checkpoint: A Databricks-managed resource group is expected. Do not place your own resources inside it.

Manual setup - create ADLS Gen2

- 1 Open Storage accounts and select Create.
- 2 Choose rg-azde-poc02, the same region, Standard performance, and LRS redundancy.
- 3 Enter a globally unique lowercase alphanumeric name such as stazdepoc02fef00m.
- 4 On the Advanced tab, enable Hierarchical namespace. This makes the account ADLS Gen2.
- 5 Keep public blob access disabled, then select Review + create and Create.

Verification

In the storage account configuration, Hierarchical namespace must be enabled before you continue.

Manual setup - container and folders

- 1 Open the storage account and go to Data storage -> Containers.
- 2 Create container poc02 with Private access.
- 3 Create or upload into raw/orders/ and raw/customers/.
- 4 Reserve separate paths for checkpoints/, schema/, and quarantine/.
- 5 Do not use the same checkpoint directory for two independent streams.

```
abfss://poc02@<storage-account>.dfs.core.windows.net/raw/orders/  
abfss://poc02@<storage-account>.dfs.core.windows.net/raw/customers/
```

Screenshot - deployed resource inventory

Home > Resource Manager | Resource groups

Resource Manager | Resource groups

Default Directory

Search

Create Manage view

Resource Manager

All resources

Favorite resources

Recent resources

Resource groups

Tags

Organization

Tools

Deployments

Help

rg-azde-poc02

Resource group

Generate Bicep code to duplicate this resource group. +2

Search

Create Manage view Delete resource group Refresh Group by none

Overview

Activity log

Access control (IAM)

Tags

Resource visualizer

Events

Settings

Cost Management

Monitoring

Automation

Help

Essentials

JSON View

Resources Recommendations

Filter for any field...

Type equals all Location equals all Add filter

Name ↑	Type	Location
ac-azde-poc02	Access Connector f...	Central India
dbw-azde-poc02	Azure Databricks S...	Central India
stazdepoc02fef00m	Storage account	Central India

The user-managed resource group contains the Access Connector, Databricks workspace, and storage account. Beginner checkpoint: These three top-level resources are the minimal Azure infrastructure required for this POC.

Manual setup - Access Connector

- 1 Search Azure Portal for Access Connector for Azure Databricks.
- 2 Select Create and choose rg-azde-poc02.
- 3 Name it ac-azde-poc02 and choose the same region.
- 4 Use a System-assigned managed identity.
- 5 After deployment, copy the complete Access Connector Resource ID for Unity Catalog.

```
/subscriptions/<subscription-id>/resourceGroups/rg-azde-poc02/  
providers/Microsoft.Databricks/accessConnectors/ac-azde-poc02
```

Manual setup - storage IAM

Give the Access Connector permission to read and write ADLS data.

- 1 Open the storage account and select Access Control (IAM).
- 2 Select Add -> Add role assignment.
- 3 Choose Storage Blob Data Contributor.
- 4 Assign access to Managed identity. Select Access Connector for Azure Databricks and choose ac-azde-poc02.
- 5 Select Review + assign. Wait several minutes for permission propagation.

Two identities

The Access Connector permission is for Databricks runtime access. Your signed-in user may also need the same data-plane role to upload files through Azure CLI.

Screenshot - IAM assignments

The screenshot displays the 'Access Control (IAM)' page for the storage account 'stazdepoc02fef00m'. The left sidebar contains navigation links: Overview, Activity log, Tags, Diagnose and solve problems, Access Control (IAM) (selected), Data migration, Events, Storage browser, Partner solutions, Resource visualizer, Data storage (expanded), Containers, Classic file shares, Queues, Tables, Security + networking, and Data management. The main content area shows the 'Role assignments' tab. At the top, it indicates 'Number of role assignments for this subscription' as 4 out of 5000. Below this is a search bar and filters for Type (All), Role (All), Scope (All scopes), and Group by (Role). The table lists three assignments: 1. Owner role assigned to user 'irshad alam' (0cabca04-45aa-440e-9ce9-4b0...) with scope 'Subscription (Inherited)'. 2. Storage Blob Data Contributor role assigned to managed identity 'ac-azde-poc02' (543c6a6f-619b-4de0-bf90-1f1...) with scope 'This resource'. 3. Storage Blob Data Contributor role assigned to user 'irshad alam' (0cabca04-45aa-440e-9ce9-4b0...) with scope 'This resource'. The bottom of the table shows 'Showing 1 - 3 of 3 results.'

Home > stazdepoc02fef00m

stazdepoc02fef00m | Access Control (IAM) ☆ ...

Storage account

Search

Overview

Activity log

Tags

Diagnose and solve problems

Access Control (IAM)

Data migration

Events

Storage browser

Partner solutions

Resource visualizer

Data storage

Containers

Classic file shares

Queues

Tables

Security + networking

Data management

Add or remove favorites by pressing Ctrl+L+Shift+F

Check access Role assignments Roles Deny assignments

Number of role assignments for this subscription ⓘ

4 5000

Search by name or email

Type : All Role : All Scope : All scopes Group by : Role

All (3) Job function roles (2) Privileged administrator roles (1)

☐	Name ↑↓	Type ↑↓	Role ↑↓	Scope ↑↓	Condition ↑↓
▼	Owner (1)				
☐	irshad alam 0cabca04-45aa-440e-9ce9-4b0...	User	Owner	Subscription (Inherited)	None
▼	Storage Blob Data Contributor (2)				
☐	ac-azde-poc02 543c6a6f-619b-4de0-bf90-1f1...	Managed identity	Storage Blob Data Contributor	This resource	Add
☐	irshad alam 0cabca04-45aa-440e-9ce9-4b0...	User	Storage Blob Data Contributor	This resource	Add

Showing 1 - 3 of 3 results.

Storage Blob Data Contributor is assigned to the Access Connector managed identity and the signed-in user. Beginner checkpoint: This enables Databricks access and local Azure CLI uploads without storage keys in notebooks.

Error fix - user role assignment failed

Symptom: Azure CLI could not find the email address in the Graph database.

Working approach: resolve the signed-in user object ID and use the ID explicitly.

```
az ad signed-in-user show --query id -o tsv

az role assignment create ^
--assignee-object-id "<returned-object-id>" ^
--assignee-principal-type "User" ^
--role "Storage Blob Data Contributor" ^
--scope "/subscriptions/<subscription-id>/resourceGroups/rg-azde-poc02/
providers/Microsoft.Storage/storageAccounts/<storage-account>"
```

Verification

List role assignments at the storage-account scope and confirm the principal type is User and the role is Storage Blob Data Contributor.

Screenshot - storage permission fixed

```
(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_02_DATABRICKS_MEDALLION_CDC_PROJECT>az role assignment create --assignee "er.irshad26@outlook.com" --role "Storage Blob Data Contributor" --scope "/subscriptions/9902e04b-fd65-40f4-86a2-1b09c1fe672c/resourceGroups/rg-azde-poc02/providers/Microsoft.Storage/storageAccounts/stazdepoc02fef00m"
Cannot find user or service principal in graph database for 'er.irshad26@outlook.com'. If the assignee is an appId, make sure the corresponding service principal is created with 'az ad sp create --id er.irshad26@outlook.com'.
```

```
(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_02_DATABRICKS_MEDALLION_CDC_PROJECT>az ad signed-in-user show --query id -o tsv
0cabca04-45aa-440e-9ce9-4b0e258604fb
```

```
(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_02_DATABRICKS_MEDALLION_CDC_PROJECT>az role assignment create --assignee-object-id "0cabca04-45aa-440e-9ce9-4b0e258604fb" --assignee-principal-type "User" --role "Storage Blob Data Contributor" --scope "/subscriptions/9902e04b-fd65-40f4-86a2-1b09c1fe672c/resourceGroups/rg-azde-poc02/providers/Microsoft.Storage/storageAccounts/stazdepoc02fef00m"
{
  "createdBy": "0cabca04-45aa-440e-9ce9-4b0e258604fb",
  "createdOn": "2026-08-31T10:46:02.603560+00:00",
  "description": "",
  "id": "/subscriptions/9902e04b-fd65-40f4-86a2-1b09c1fe672c/resourceGroups/rg-azde-poc02/providers/Microsoft.Storage/storageAccounts/stazdepoc02fef00m/providers/Microsoft.Authorization/roleAssignments/a9b48e68-8e9c-8ce2-2557-c92bf01c7ae9",
  "name": "a9b48e68-8e9c-8ce2-2557-c92bf01c7ae9",
  "principalId": "0cabca04-45aa-440e-9ce9-4b0e258604fb",
  "principalName": "er.irshad26_outlook.com#EXT#@erirshad26outlook.onmicrosoft.com",
  "principalType": "User",
  "resourceGroup": "rg-azde-poc02",
  "roleDefinitionId": "/subscriptions/9902e04b-fd65-40f4-86a2-1b09c1fe672c/providers/Microsoft.Authorization/roleDefinitions/ba92f5b4-2d11-453d-a403-e96b0029c9fe",
  "roleDefinitionName": "Storage Blob Data Contributor",
  "scope": "/subscriptions/9902e04b-fd65-40f4-86a2-1b09c1fe672c/resourceGroups/rg-azde-poc02/providers/Microsoft.Storage/storageAccounts/stazdepoc02fef00m",
  "type": "Microsoft.Authorization/roleAssignments",
  "updatedBy": "0cabca04-45aa-440e-9ce9-4b0e258604fb",
  "updatedOn": "2026-08-31T10:46:02.603560+00:00"
}
```

```
(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_02_DATABRICKS_MEDALLION_CDC_PROJECT>
```

The email-based assignment failed; using the signed-in user object ID created the role assignment successfully. Beginner checkpoint: When an external or guest-style user principal cannot be resolved by email, use the object ID returned by the signed-in-user command.

Manual setup - Azure Databricks workspace

- 1 Search Azure Portal for Azure Databricks and select Create.
- 2 Choose rg-azde-poc02, the same region, and name the workspace dbw-azde-poc02.
- 3 Use Premium when the required Unity Catalog capabilities depend on that tier.
- 4 Enable No Public IP when supported by the account and lab design.
- 5 Create the workspace, wait for deployment, and select Launch Workspace.

Verification

The workspace Overview page should show Active status, the expected managed resource group, and the workspace URL.

Screenshot - Databricks workspace

The screenshot displays the Azure Portal interface for an Azure Databricks service. The main heading is 'dbw-azde-poc02', identified as an 'Azure Databricks Service'. The left sidebar contains a navigation menu with options like 'Overview', 'Activity log', 'Access control (IAM)', 'Tags', 'Diagnose and solve problems', 'Resource visualizer', 'Settings', 'Monitoring', 'Automation', and 'Help'. The 'Overview' section is active, showing 'Essentials' such as Status (Active), Resource group (rg-azde-poc02), Location (Central India), Subscription (Azure subscription 1), and Subscription ID (9902e04b-fd65-40f4-86a2-1b09c1fe672c). A 'Tags' section lists 'environment : lab', 'managed_by : terraform', and 'project : POC-02-Databricks-Medallion-CDC'. On the right, a 'Workspace type' section shows 'Hybrid', 'Managed Resource Group' (rg-azde-poc02-databricks-managed), 'URL' (https://adb-7405616874846049.9.azuredatabricks.net), 'Pricing Tier' (Premium (+ Role-based access controls)), and 'Enable No Public IP' (Yes). A large red Databricks logo is centered at the bottom, with a 'Launch Workspace' button below it. The top of the page includes a search bar and a 'Delete' button. The bottom of the page has a footer note: 'Add or remove favorites by pressing Ctrl+L+Shift+F'.

Azure Portal shows the completed Databricks workspace, Premium pricing tier, managed resource group, and Terraform tags.

Beginner checkpoint: The workspace exists in the same region as the POC resources and is ready for Unity Catalog configuration.

Unity Catalog - storage credential

- 1 In Databricks, open Catalog and choose the option to add a storage credential.
- 2 Name it poc02_storage_cred.
- 3 Choose Azure Managed Identity.
- 4 Paste the Access Connector Resource ID.
- 5 Create and validate the credential when the UI provides a validation action.

Security design

Notebooks receive ADLS access through the managed identity. Do not place storage account keys or Databricks tokens in notebook source.

Unity Catalog - external location

- 1 Open Catalog -> Add -> External location.
- 2 Name it poc02_ext.
- 3 Set the URL to the root of the private poc02 container.
- 4 Choose poc02_storage_cred.
- 5 Grant the intended POC user or group the minimum required file and managed-storage privileges.

```
abfss://poc02@<storage-account>.dfs.core.windows.net/
```

Typical privileges

READ FILES, WRITE FILES, CREATE EXTERNAL TABLE, and CREATE MANAGED STORAGE may be needed depending on where the tables are created.

Unity Catalog - catalog and schemas

```
CREATE CATALOG IF NOT EXISTS azde_poc;  
CREATE SCHEMA IF NOT EXISTS azde_poc.bronze;  
CREATE SCHEMA IF NOT EXISTS azde_poc.silver;  
CREATE SCHEMA IF NOT EXISTS azde_poc.gold;  
CREATE SCHEMA IF NOT EXISTS azde_poc.quarantine;
```

Use the three-part namespace `catalog.schema.table` in every notebook and validation query.

Verification

Catalog Explorer should show bronze, silver, gold, quarantine, and the system-generated `information_schema`.

Screenshot - Unity Catalog available

The screenshot shows the Databricks Unity Catalog interface. The left sidebar contains a navigation menu with options like 'New', 'Workspace', 'Recents', 'Catalog', 'Jobs & Pipelines', 'Compute', 'Discover', 'Marketplace', 'SQL', 'SQL Editor', 'Queries', 'Dashboards', 'Genie Agents', 'Alerts', 'Query History', 'SQL Warehouses', 'Data Engineering', 'Runs', 'Data Ingestion', and 'Visual Data Prep'. The 'Catalog' section is selected, showing a search bar and a list of catalogs under 'My organization' and 'Shares received'. The 'azde_poc' catalog is selected, displaying a list of suggested catalog objects. The main panel shows a table of suggested objects with columns for Name, Reason for suggestion, and Type.

Name	Reason for suggestion	Type
media_customer_reviews samples.bakehouse	Quick start with samples data	Table
sales_customers samples.bakehouse	Quick start with samples data	Table
sales_suppliers samples.bakehouse	Quick start with samples data	Table
media_gold_reviews_chunked samples.bakehouse	Quick start with samples data	Table
sales_franchises samples.bakehouse	Quick start with samples data	Table
sales_transactions samples.bakehouse	Quick start with samples data	Table

Catalog Explorer shows the azde_poc catalog in the Databricks workspace.
Beginner checkpoint: The workspace can browse governed catalog objects through Unity Catalog.

Screenshot - Medallion schemas

The screenshot displays the Databricks interface. On the left, the 'Catalog' sidebar shows a tree view with 'My organization' expanded, containing 'dbw_azde_poc02', 'system', and 'azde_poc'. The 'azde_poc' catalog is selected, showing its sub-catalogs: 'bronze', 'gold', 'information_schema', 'quarantine', and 'silver'. The 'information_schema' catalog is further expanded, showing 'No data' for each of its sub-catalogs. The main panel shows the 'Catalog Explorer' for 'azde_poc'. The 'Overview' tab is active, displaying the catalog's description: 'POC-02 Medallion catalog managed by Terraform'. Below the description, a table lists the schemas in the catalog. The table has three columns: 'Name', 'Owner', and 'Created at'. The schemas listed are 'bronze', 'gold', 'information_schema', 'quarantine', and 'silver', all created on August 31, 2026, at 04:20 PM. The 'Owner' for 'information_schema' is 'System user', while the others are 'er.irshad26@outlook.com'. The 'About this catalog' section shows the owner as 'er.irshad26@outlook.com'. The 'Tags' section has an 'Add tags' button. The 'Policies' section is also visible.

Name	Owner	Created at
bronze	er.irshad26@outlook.com	Aug 31, 2026, 04:20 PM
gold	er.irshad26@outlook.com	Aug 31, 2026, 04:20 PM
information_schema	System user	Aug 31, 2026, 04:20 PM
quarantine	er.irshad26@outlook.com	Aug 31, 2026, 04:20 PM
silver	er.irshad26@outlook.com	Aug 31, 2026, 04:20 PM

The azde_poc catalog contains bronze, silver, gold, quarantine, and information_schema. Beginner checkpoint: The catalog description and schema inventory confirm the Terraform-created logical structure.

Compute setup - low-cost single node

Use serverless compute when available, or create a small single-node cluster for this tiny sample dataset.

```
num_workers = 0

spark_conf = {
    "spark.databricks.cluster.profile" = "singleNode"
    "spark.master" = "local[*]"
}

custom_tags = {
    "ResourceClass" = "SingleNode"
}
```

- Set auto-termination to 15 minutes.
- Use the smallest compatible node type available in the region.
- Confirm Unity Catalog data access is enabled for the selected runtime.

Error fix - regional vCPU quota exceeded

Observed failure: Central India had a Total Regional Cores limit of 4, while the original driver plus worker configuration required 8 vCPUs.

Choice	Action
Recommended for this POC	Switch to single-node compute with zero workers and rerun Terraform apply.
Alternative	Choose smaller compatible VMs so total driver and worker cores stay within quota.
For real multi-node work	Request increases for Total Regional vCPUs and the matching VM family, then repair the job run.

Diagnosis rule

A cluster event that fails before Spark starts is usually an Azure capacity, policy, image, or quota issue - not a notebook logic issue.

Screenshot - first Phase 1 job failed

The screenshot shows the Databricks interface for a job configuration. The job name is "job-294354939284236-run-832485362905592-poc02_small_cluster". The job status is "Failed" with a red error icon. The error message is "Cluster terminated due to error" with the error code "AZURE_QUOTA_EXCEEDED_EXCEPTION". The message states: "The VM size you are specifying is not available. [details] QuotaExceeded: Operation could not be completed ...". A yellow warning banner indicates: "Based on your existing cloud quotas, your default compute size has been reduced. View or increase your quotas in Azure portal." The configuration page shows the following details:

- General**
 - Compute name: job-294354939284236-run-832485362905592-poc02_
 - Policy: Unrestricted
- Performance**
 - Machine learning: ☐
 - Databricks runtime: 18 LTS (Scala 2.13, Spark 4.1.0)
 - Photon acceleration: ☐
- Summary**
 - 16 GB Memory
 - 4 Cores
- Data access**
 - Unity Catalog
- Price**
 - 2 DBU/h

The job compute terminated with `AZURE_QUOTA_EXCEEDED_EXCEPTION` before notebook processing could begin.

Beginner checkpoint: The error was resolved by changing the job cluster to a single-node design that fit the subscription quota.

Import the six Databricks notebooks

- 1 Open Workspace and create a folder named POC_02_DATABRICKS_MEDALLION_CDC.
- 2 Import 00_setup.py and 01_bronze_ingest.py.
- 3 Import 02_silver_quality.py and 03_gold_dimensions.py.
- 4 Import 04_cdf_consumer.py and 05_performance_governance.py.
- 5 Confirm each item is recognized as a Python notebook and points to the intended workspace folder.

Terraform path

When notebook resources are defined in Terraform, terraform apply uploads these files automatically.

Screenshot - notebooks imported

The screenshot displays the Databricks interface within the Microsoft Azure environment. The left sidebar contains navigation options such as 'New', 'Workspace', 'Recents', 'Catalog', 'Jobs & Pipelines', 'Compute', 'Discover', 'Marketplace', 'SQL', 'SQL Editor', 'Queries', 'Dashboards', 'Genie Agents', 'Alerts', 'Query History', 'SQL Warehouses', 'Data Engineering', 'Runs', 'Data Ingestion', 'Visual Data Prep', and 'AI/ML'. The main workspace area is titled 'Workspace > Shared > POC_02_DATABRICKS_MEDALLION_CDC'. It features a search bar and filters for 'Type', 'Owner', and 'Last modified'. A table lists six notebooks, all of which are 'Notebook' type and owned by 'er.irshad26@outl...'. The notebooks are: '00_setup', '01_bronze_ingest', '02_silver_quality', '03_gold_dimensions', '04_cdf_consumer', and '05_performance_governance'. All notebooks were created and last updated on 'Aug 31, 2026, 04:20 PM'.

Name	Type	Owner	Created at	Last updated at
00_setup	Notebook	er.irshad26@outl...	Aug 31, 2026, 04:20 PM	Aug 31, 2026, 04:20 PM
01_bronze_ingest	Notebook	er.irshad26@outl...	Aug 31, 2026, 04:20 PM	Aug 31, 2026, 04:20 PM
02_silver_quality	Notebook	er.irshad26@outl...	Aug 31, 2026, 04:20 PM	Aug 31, 2026, 04:20 PM
03_gold_dimensions	Notebook	er.irshad26@outl...	Aug 31, 2026, 04:20 PM	Aug 31, 2026, 04:20 PM
04_cdf_consumer	Notebook	er.irshad26@outl...	Aug 31, 2026, 04:20 PM	Aug 31, 2026, 04:20 PM
05_performance_governance	Notebook	er.irshad26@outl...	Aug 31, 2026, 04:20 PM	Aug 31, 2026, 04:20 PM

The Databricks workspace folder contains all six POC notebooks. Beginner checkpoint: Setup, Bronze, Silver, Gold, CDF, and performance/governance code is available before jobs are executed.

Phase 1 - upload the initial files

Upload only the Phase 1 files. Keep Phase 2 outside ADLS until the first run is complete and validated.

```
az storage fs file upload ^  
--account-name <storage-account> ^  
--file-system poc02 ^  
--path raw/orders/orders_batch_001.csv ^  
--source sample_data\phase1\raw\orders\orders_batch_001.csv ^  
--auth-mode login --overwrite true  
  
az storage fs file upload ^  
--account-name <storage-account> ^  
--file-system poc02 ^  
--path raw/customers/customers_batch_001.csv ^  
--source sample_data\phase1\raw\customers\customers_batch_001.csv ^  
--auth-mode login --overwrite true
```


Phase 1 - parameters and run order

Widget	Value
storage_account	<unique ADLS account>
container	poc02
catalog	azde_poc
batch_id	phase1

```
00_setup.py
-> 01_bronze_ingest.py
-> 02_silver_quality.py
-> 03_gold_dimensions.py
```

Expected job status

All four tasks must show Succeeded before Phase 2 data is uploaded.

Phase 1 - what each notebook does

Notebook	Responsibility
00_setup	Validates paths and creates the catalog/schema foundation.
01_bronze_ingest	Auto Loader reads new CSV files and adds ingestion metadata.
02_silver_quality	Casts types, validates records, writes quarantine, and deduplicates.
03_gold_dimensions	MERGE builds fact_orders, SCD1 dim_product, and SCD2 dim_customer; CDF is enabled.

Idempotency test

Record Bronze counts, rerun Bronze without uploading a new file, and confirm the counts do not increase.

Screenshot - Phase 1 succeeded

The screenshot displays the Databricks workspace interface. The top navigation bar includes the Microsoft Azure logo, the Databricks logo, a search bar, and the user profile 'dbw-azde-poc02'. The left sidebar contains navigation options: New, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, SQL Warehouses, Data Engineering, **Runs** (highlighted), Data Ingestion, and Visual Data Prep. The main content area shows the 'Jobs & Pipelines' view for the 'POC02-Phase1-Medallion' job. The job run 'POC02-Phase1-Medallion run' is displayed in the 'Graph' view, showing a sequence of four tasks: 'setup', 'bronze', 'silver', and 'gold'. Each task is marked as 'Succeeded' with a green checkmark. The 'gold' task is the final task in the pipeline. To the right of the graph, the 'Job run details' panel provides information about the job run, including the Job ID, Job run ID, Launched status, Started and Ended times, Duration (7m 55s), Status (Succeeded), and Lineage (2 upstream tables, 1 downstream table). Below the job run details, the 'Compute' section shows the cluster 'poc02_small_cluster' as 'Cluster terminated'. The bottom right corner of the interface shows the AI/ML section.

Microsoft Azure databricks

Search data, notebooks, recents, and more... CTRL + P

dbw-azde-poc02

New

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Jobs & Pipelines > POC02-Phase1-Medallion >

POC02-Phase1-Medallion run Send feedback

Delete job run Repair run

Graph Timeline List

setup Succeeded - 6m 20s
_ASBROCKS_MEDALLION_CDC/00_setup
poc02_small_cluster

bronze Succeeded - 31s
_MEDALLION_CDC/01_bronze_ingest
poc02_small_cluster

silver Succeeded - 37s
_S_MEDALLION_CDC/02_silver_quality
poc02_small_cluster

gold Succeeded - 23s
_EDALLION_CDC/03_gold_dimensions
poc02_small_cluster

Job run details

Job ID 294354939284236

Job run ID 76638568125137

Launched Manually

Started Sep 01, 2026, 05:12 PM

Ended Sep 01, 2026, 05:20 PM

Duration 7m 55s

Status Succeeded

Lineage 2 upstream tables, 1 downstream table

View run events

Compute

Cluster terminated

poc02_small_cluster

Single node: Standard_D4ads_v5 · Release: 18.3.5

View details Spark UI Logs Metrics

Setup, Bronze, Silver, and Gold tasks all completed successfully after the compute change. Beginner checkpoint: The initial Medallion pipeline ran end to end; lineage information is also visible in the job details.

Auto Loader checkpoint verification

- 1 Run a Bronze row-count query after Phase 1.
- 2 Do not add or rename any input file.
- 3 Rerun `01_bronze_ingest.py`.
- 4 Run the same count query again.
- 5 The count should remain unchanged because processed files are recorded in the checkpoint.

Do not test by copying the same data under a new filename

Auto Loader tracks file discovery. A duplicate payload with a new filename can be treated as a new source file; business-key deduplication must protect Silver and Gold.

Phase 2 - upload incremental data

Phase 2 introduces new and changed business records plus a new nullable order column named sales channel.

```
az storage fs file upload ^  
--account-name <storage-account> ^  
--file-system poc02 ^  
--path raw/orders/orders_batch_002_schema_evolution.csv ^  
--source sample_data\phase2\raw\orders\  
orders_batch_002_schema_evolution.csv ^  
--auth-mode login --overwrite true
```

```
az storage fs file upload ^  
--account-name <storage-account> ^  
--file-system poc02 ^  
--path raw/customers/customers_batch_002_customer_change.csv ^  
--source sample_data\phase2\raw\customers\  
customers_batch_002_customer_change.csv ^  
--auth-mode login --overwrite true
```

Phase 2 - run order and expected changes

```
01_bronze_ingest.py batch_id = phase2  
-> 02_silver_quality.py  
-> 03_gold_dimensions.py  
-> 04_cdf_consumer.py
```

- Auto Loader discovers only the Phase 2 files.
- The nullable sales_channel column demonstrates controlled schema evolution.
- Delta MERGE applies new and updated orders.
- Customer C002 changes city from Mumbai to Bengaluru, creating SCD2 history.
- CDF writes changed fact rows to fact_orders_changes_audit.

Schema evolution - expected restart

With Auto Loader configured for `addNewColumns`, discovering `sales_channel` can update schema metadata and stop the first attempt.

- 1 Confirm the error is specifically the expected new-column/schema-update event.
- 2 Inspect the inferred schema metadata rather than deleting checkpoints.
- 3 Rerun the Bronze task once.
- 4 Verify `sales_channel` exists and older records contain null when the field was absent.
- 5 Rerun Silver and Gold so the controlled schema change reaches curated layers.

Production rule

Do not allow unrestricted schema drift. New fields should be reviewed for type, nullability, downstream compatibility, and data-contract impact.

Screenshot - Phase 2 succeeded

The screenshot displays the Databricks workspace interface. The top navigation bar includes the Microsoft Azure logo, the Databricks logo, a search bar, and the user profile 'dbw-azde-poc02'. The left sidebar shows a navigation menu with categories like 'New', 'Workspace', 'SQL', 'Data Engineering', and 'AI/ML'. The 'Runs' section is highlighted under 'Data Engineering'.

The main content area shows the 'Jobs & Pipelines' view for the pipeline 'POC02-Phase2-Incremental-CDC'. The 'Graph' tab is selected, showing a sequence of four tasks: 'bronze_incremental', 'silver_refresh', 'gold_merge_scd', and 'cdf_consumer'. Each task is marked as 'Succeeded' with a green checkmark. The tasks are connected by arrows, indicating a sequential flow.

On the right side, the 'Job run details' panel is expanded, showing the following information:

- Job ID: [547011624753363](#)
- Job run ID: [862364884820723](#)
- Launched: Manually
- Started: Sep 01, 2026, 05:21 PM
- Ended: Sep 01, 2026, 05:25 PM
- Duration: 3m 33s
- Status: ✔ Succeeded
- Lineage: 1 upstream table, 3 downstream tables

Below the job run details, the 'Compute' section shows the cluster 'poc02_small_cluster' is terminating. The cluster configuration is 'Standard_D4ads_v5 · Release: 18.3.5'. Links for 'View details', 'Spark UI', 'Logs', and 'Metrics' are provided.

Bronze incremental, Silver refresh, Gold MERGE/SCD, and CDF consumer tasks all succeeded.
Beginner checkpoint: The incremental pipeline completed on a single-node Standard_D4ads_v5 cluster.

Validation SQL - layer counts

```
SELECT 'bronze.orders' AS table_name, COUNT(*) AS total_rows
FROM azde_poc.bronze.orders
UNION ALL
SELECT 'bronze.customers', COUNT(*)
FROM azde_poc.bronze.customers
UNION ALL
SELECT 'silver.orders', COUNT(*)
FROM azde_poc.silver.orders
UNION ALL
SELECT 'silver.customers', COUNT(*)
FROM azde_poc.silver.customers;
```

Interpretation

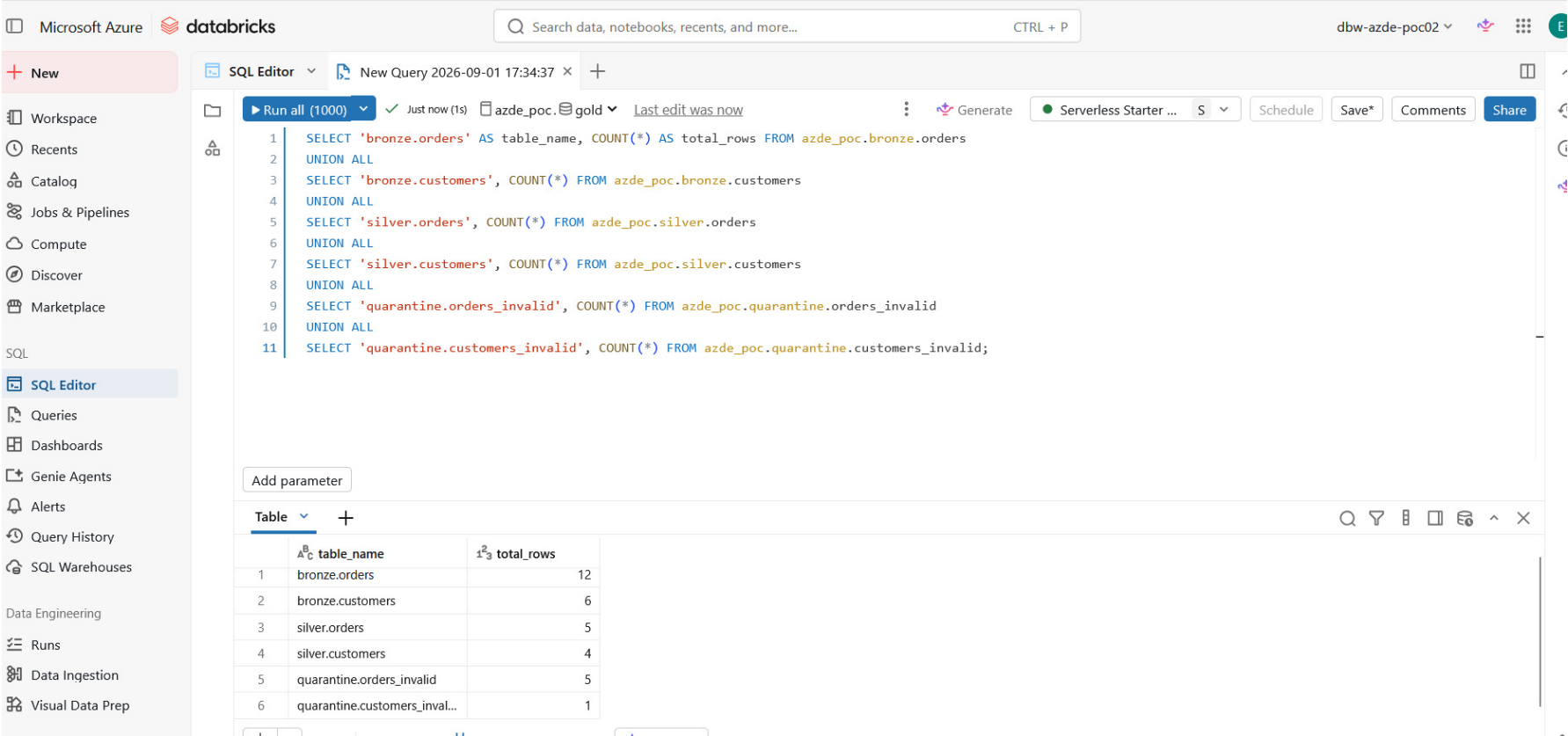
Bronze includes accepted source rows before Silver business deduplication and rejection routing. Silver should contain only valid, standardized current records.

Validation SQL - quarantine counts

```
SELECT 'quarantine.orders_invalid' AS table_name, COUNT(*) AS total_rows
FROM azde_poc.quarantine.orders_invalid
UNION ALL
SELECT 'quarantine.customers_invalid', COUNT(*)
FROM azde_poc.quarantine.customers_invalid;
```

Observed table	Rows
Bronze orders	9
Bronze customers	6
Silver orders	5
Silver customers	4
Quarantine orders	5
Quarantine customers	1

Screenshot - final table counts



The screenshot displays the Databricks SQL Editor interface. The top navigation bar includes the Microsoft Azure logo, the Databricks logo, a search bar, and the user profile 'dbw-azde-poc02'. The left sidebar shows a navigation menu with options like 'New', 'Workspace', 'Recents', 'Catalog', 'Jobs & Pipelines', 'Compute', 'Discover', 'Marketplace', 'SQL Editor' (selected), 'Queries', 'Dashboards', 'Genie Agents', 'Alerts', 'Query History', 'SQL Warehouses', 'Data Engineering', 'Runs', 'Data Ingestion', and 'Visual Data Prep'. The main editor area shows a SQL query with 11 lines, grouped into 6 rows by the 'Run all (1000)' button. The query counts rows for various tables in the 'azde_poc' database. The results table at the bottom shows the following data:

Table	table_name	total_rows
1	bronze.orders	12
2	bronze.customers	6
3	silver.orders	5
4	silver.customers	4
5	quarantine.orders_invalid	5
6	quarantine.customers_inval...	1

Databricks SQL shows the observed Bronze, Silver, and quarantine row counts.
Beginner checkpoint: The final counts match the expected transformation and quality-routing behavior after both phases.

Validation SQL - quality reasons

```
SELECT error_reason, COUNT(*) AS record_count
FROM azde_poc.quarantine.orders_invalid
GROUP BY error_reason
ORDER BY error_reason;
```

Quality failure	Rows
order_id is null or blank	1
quantity must be greater than 0	1
unit_price must be at least 0	1
status is not allowed	1
invalid order_ts	1

Screenshot - quarantine reasons

The screenshot shows the Databricks SQL Editor interface. The top navigation bar includes the Microsoft Azure logo, the Databricks logo, a search bar, and the user profile 'dbw-azde-poc02'. The left sidebar contains a 'New' button and a list of navigation items: Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor (selected), Queries, Dashboards, Genie Agents, Alerts, Query History, and SQL Warehouses. The main area displays a SQL query in the 'SQL Editor' tab:

```
1 SELECT error_reason, COUNT(*) AS record_count
2 FROM azde_poc.quarantine.orders_invalid
3 GROUP BY error_reason;
```

Below the query editor, the results are shown in a table format. The table has two columns: 'error_reason' and 'record_count'. It contains five rows of data, each representing a different error reason and its frequency.

	error_reason	record_count
1	order_id is null/blank	1
2	quantity must be > 0	1
3	unit_price must be >= 0	1
4	status is not allowed	1
5	invalid order_ts	1

At the bottom of the results table, there is a download icon, a dropdown menu, and the text '5 rows | 0.088s runtime | See performance (1) | Optimize'. The bottom right corner of the interface shows 'Refreshed Now' and a refresh icon.

The query returns one row for each of the five intentionally tested order-quality failures. Beginner checkpoint: The quarantine layer proves that bad records were preserved for investigation instead of silently discarded.

Validation - SCD Type 2 history

```
SELECT customer_id, city, effective_from, effective_to, is_current
FROM azde_poc.gold.dim_customer
WHERE customer_id = 'C002'
ORDER BY effective_from;
```

Version	City	End date	Current
Old version	Mumbai	Closed effective_to	false
New version	Bengaluru	NULL effective_to	true

SCD2 rule

A changed customer creates a new row and closes the prior current row. History is retained instead of overwriting the previous value.

Validation - Change Data Feed

The Gold fact table has table Change Data Feed enabled. The consumer reads row changes rather than rescanning the complete fact table.

```
SELECT _change_type, _commit_version, _commit_timestamp, *  
FROM table_changes(  
  "azde_poc.gold.fact_orders",  
  <starting-version>  
)  
ORDER BY _commit_version, order_id;
```

- Typical change types include insert, update_preimage, update_postimage, and delete.
- The consumer writes audit rows into azde_poc.gold.fact_orders_changes_audit.
- Store the last successfully processed commit version for a production incremental consumer.

Screenshot - Gold customer dimension

The screenshot displays the Databricks interface. On the left, the 'Catalog' sidebar shows a tree structure under 'My organization' with folders for 'dbw_azde_poc02', 'system', 'azde_poc', 'bronze', and 'gold'. The 'gold' folder is expanded, showing a list of tables: 'dim_customer', 'dim_product', 'fact_orders', and 'fact_orders_changes_audit'. The 'dim_customer' table is selected. The main panel, 'Catalog Explorer', shows the 'dim_customer' table details. It includes tabs for 'Overview', 'Sample Data', 'Details', 'Permissions', 'Policies', 'History', 'Lineage', 'Insights', and 'Quality'. The 'Sample Data' tab is active, displaying a table with 4 rows of sample data. Above the table, there is a search bar and a 'Reset' button. Below the table, there are three sample queries: 'How many current customers are in each country?', 'Which cities have the most current customers?', and 'Are there customers with missing email?'. The top of the interface shows the 'Microsoft Azure' logo, the 'databricks' logo, a search bar, and the user profile 'dbw-azde-poc02'.

	customer_sk	customer_id	customer_name	email	
1	8df003dcbfdd1fb4aef9540182f2a0646b922b3b0b4f091dbaa0650c4e9233aa	C001	Asha Khan	asha@example.com	T
2	db01268e5ae45ee63d627c348c52e91a52e2fc5e57596899840dcb7bc80d4bfc	C002	Ravi Mehta	ravi@example.com	B
3	e8fa9c2e0fb78856089cd4ceb5e5bd30feed7f7b8c4b17763147223ecb72311b	C003	Neha Singh	neha@example.com	P
4	f164a03ea2a6daed0d3fa13310af944e18bba38f810652a4d620c7ada0a3917d	C004	Imran Ali	imran@example.com	C

Catalog Explorer shows the Gold tables and sample rows from dim_customer.
Beginner checkpoint: The Gold layer contains dim_customer, dim_product, fact_orders, and fact_orders_changes_audit.

Terraform-first workflow

```
00_check_prerequisites
-> 01_configure_terraform
-> 02_terraform_apply
-> 03_upload_phase1
-> 04_open_phase1_job
-> 05_upload_phase2
-> 06_open_phase2_job
-> 08_show_status
-> 07_destroy
```

Ownership rule

When Terraform creates the lab, Terraform must also destroy it. Avoid manual portal deletion of Terraform-owned resources.

Terraform variables

```
cd POC_02_DATABRICKS_MEDALLION_CDC_TERRAFORM_PROJECT
copy terraform\terraform.tfvars.example terraform\terraform.tfvars
notepad terraform\terraform.tfvars
```

Variable	Value or rule
subscription_id	Selected Azure subscription
resource_group_name	rg-azde-poc02
location	Region with suitable quota
storage_account_name	Globally unique name
databricks_workspace_name	dbw-azde-poc02
environment	lab or dev

Terraform initialize, validate, plan, and apply

```
cd terraform
terraform fmt -check
terraform init
terraform validate
terraform plan -out=tfplan
terraform show tfplan
terraform apply tfplan
terraform output
```

- Read the plan before applying it.
- Confirm only the POC resource group, storage, Access Connector, RBAC, Databricks, Unity Catalog objects, notebooks, and jobs are targeted.
- Commit `.terraform.lock.hcl` for repeatable provider selection.

Screenshot - Terraform initialization example

```
(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_01_secure_batch_adf_adls_sql\infra\terraform>terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/azurerm versions matching "~> 4.0"...
- Installing hashicorp/azurerm v4.81.0...
- Installed hashicorp/azurerm v4.81.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_01_secure_batch_adf_adls_sql\infra\terraform>terraform plan
var.data_factory_name
  Globally unique Data Factory name.

Enter a value:
```

Terraform successfully installed the AzureRM provider and created the dependency lock file. Beginner checkpoint: The visible command path belongs to an earlier POC. For POC-02, run the same lifecycle from its own terraform directory and prepare terraform.tfvars first.

Terraform provider responsibilities

Provider	Resources
AzureRM provider	Resource group, ADLS Gen2, file system, Access Connector, role assignment, and Databricks workspace
Databricks provider	Storage credential, external location, catalog, schemas, notebook imports, and Phase 1/Phase 2 jobs

Resource group -> Storage -> Access Connector -> RBAC
Databricks workspace -> Credential -> External location
Catalog -> Schemas -> Notebooks -> Jobs

Dependency rule

Workspace-level Databricks resources cannot be created until the Azure workspace exists and the provider can authenticate to its URL.

Numbered CMD scripts

```
cmd\00_check_prerequisites.cmd  
cmd\01_configure_terraform.cmd  
cmd\02_terraform_apply.cmd  
cmd\03_upload_phase1.cmd  
cmd\04_open_phase1_job.cmd  
cmd\05_upload_phase2.cmd  
cmd\06_open_phase2_job.cmd  
cmd\08_show_status.cmd
```

Why use the scripts

They make the beginner workflow repeatable and reduce mistakes in long Terraform, Azure CLI, and workspace-opening commands.

Terraform state and Git safety

Category	Rule
Commit	Terraform source, notebooks, sample data, SQL, documentation, tfvars.example, and .terraform.lock.hcl
Do not commit	.terraform/, terraform.tfstate*, tfplan files, destroy plans, private terraform.tfvars, tokens, secrets, or CLI profiles
Team environment	Use a protected remote Azure Storage backend with restricted access and state locking.

State contains infrastructure metadata

Treat state as sensitive operational data even when no explicit secret was intentionally added.

Terraform destroy

Use the project safety wrapper after all validation is complete.

```
cmd\07_destroy.cmd
```

Manual equivalent:

```
cd terraform
terraform plan -destroy -out=destroy.tfplan
terraform show destroy.tfplan
terraform apply destroy.tfplan
```

Safety check

The wrapper asks you to type DESTROY before deletion. Review the destroy plan and confirm it targets only POC-02.

Screenshot - Terraform destroy completed

```
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 02m10s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 02m20s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 02m30s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 02m40s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 02m50s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 03m00s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 03m10s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 03m20s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 03m30s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 03m40s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 03m50s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 04m00s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 04m10s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 04m20s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 04m30s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 04m40s elapsed]
azurerm_databricks_workspace.poc02: Destruction complete after 4m41s
azurerm_resource_group.poc02: Destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-1b09c1fe672c/resourceGroups/rg-azde-poc02]
azurerm_resource_group.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-1b09c1fe672c/resourceGroups/rg-azde-poc02, 00m10s elapsed]
azurerm_resource_group.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-1b09c1fe672c/resourceGroups/rg-azde-poc02, 00m20s elapsed]
azurerm_resource_group.poc02: Destruction complete after 24s

Destroy complete! Resources: 10 destroyed.

(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_02_DATABRICKS_MEDALLION_CDC_PROJECT\terraform>
```

Terraform reports that 10 resources were destroyed after the Databricks workspace finished deleting. Beginner checkpoint: Workspace deletion can take several minutes. Wait for Terraform to report completion before closing CMD.

Post-destroy verification

```
az group exists --name rg-azde-poc02
az group list --output table
az resource list ^
--resource-group rg-azde-poc02 ^
--output table
```

- The POC resource group should no longer exist after deletion completes.
- The Databricks-managed resource group may remain briefly while Azure finalizes cleanup.
- If Terraform created the resources, do not use `az group delete` before Terraform destroy.

Repeat-run readiness

A clean destroy lets the same Terraform project recreate the lab later with predictable state.

Screenshot - Azure CLI cleanup checks

```
Command Prompt - az group X + v
(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_02_DATABRICKS_MEDALLION_CDC_PROJECT\terraform>az group list --output table
Name            Location      Status
-----
rg-azde-poc02   eastus        Succeeded

(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_02_DATABRICKS_MEDALLION_CDC_PROJECT\terraform>az resource list --resource-group rg-azde-poc02 --output table
Name            ResourceGroup Location      Type                                Status
-----
stazdepoc02     rg-azde-poc02 eastus        Microsoft.Storage/storageAccounts Succeeded

(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_02_DATABRICKS_MEDALLION_CDC_PROJECT\terraform>az group delete --name rg-azde-poc02 --yes
- InProgress ..
```

Azure CLI lists the resource group and resources before a cleanup command is started.
Beginner checkpoint: Use these listing commands for verification, but use Terraform destroy as the authoritative deletion path for Terraform-owned resources.

Troubleshooting summary

Problem	Working fix
az command missing	Install Azure CLI, reopen CMD, verify with where az.
Role assignment email not found	Resolve the signed-in user object ID and use --assignee-object-id.
Storage upload unauthorized	Grant the user Storage Blob Data Contributor and wait for RBAC propagation.
vCPU quota exceeded	Use single-node zero-worker compute or request a quota increase.
Auto Loader detects new column	Review schema metadata and rerun Bronze once for the controlled new field.
Terraform prompts for values	Create and complete terraform.tfvars before plan.

Performance and governance review

- Partitions: avoid excessive partition counts on tiny data; at scale, partition by selective, commonly filtered columns only.
- Small files: compact output files and control write parallelism because many small files increase metadata and scheduling overhead.
- Joins: inspect query plans and use broadcast joins only when the dimension is safely small.
- Delta maintenance: evaluate OPTIMIZE and clustering strategies based on real query patterns, not this tiny benchmark.
- Unity Catalog: review ownership, least-privilege grants, comments, tags, and lineage.
- At 100 GB or 1 TB: use production clusters, job retries, monitoring, expectations, data contracts, and protected checkpoint/state storage.

Interview review - core concepts

Question	Short answer
Bronze vs Silver vs Gold	Raw/audited source, validated/standardized data, and business-ready facts/dimensions.
Auto Loader vs file listing	Auto Loader incrementally discovers files and manages progress/schema metadata at scale.
MERGE vs overwrite	MERGE applies key-based inserts and updates; overwrite replaces a target scope.
Checkpoint	Stream progress and discovered-file state, separate from business tables.
CDF vs CDC	CDC is the pattern; Delta CDF is the table feature exposing row-level changes.

Interview review - dimensions and scale

Question	Short answer
SCD Type 1	Update the current value without preserving prior versions.
SCD Type 2	Close the old row and insert a new current row with effective dates.
Why quarantine	Preserve invalid records for investigation, correction, and replay.
Why small files hurt Spark	They increase listing, task scheduling, open/close, and metadata overhead.
Why Unity Catalog	Centralized naming, permissions, ownership, metadata, and lineage.

Final completion checklist

- [PASS] Resource group, ADLS Gen2, Access Connector, IAM, and Databricks workspace created.
- [PASS] Unity Catalog azde_poc with Bronze, Silver, Gold, and quarantine schemas created.
- [PASS] Six notebooks imported and two job workflows available.
- [PASS] Phase 1 setup, Bronze, Silver, and Gold tasks succeeded.
- [PASS] Checkpoint behavior prevented unchanged files from being processed again.
- [PASS] Phase 2 schema evolution, MERGE, SCD2, and CDF tasks succeeded.
- [PASS] Final Bronze, Silver, Gold, quarantine, and quality-rule validation completed.
- [PASS] Terraform reported 10 resources destroyed.

POC status

The supplied run evidence confirms that POC-02 was completed end to end, including the real IAM and quota fixes, final data validation, and cleanup.